

Guion — Microservicio de Órdenes de Compra

Duración sugerida: 12-15 minutos. Puede dividirse en 2 videos si es necesario.

Bloque naranja = qué hacer / dónde hacer clic. **Bloque azul** = qué decir en voz alta.

0. Prepara tus pestañas antes de grabar

QUÉ HACER (antes de grabar):

1. Abre 3 pestañas del navegador, en este orden: (1) tu sitio de Netlify, (2) MongoDB Atlas ya con sesión iniciada en tu cluster, (3) Swagger de Order.API (/swagger/index.html).
2. Abre VS Code con el proyecto, y deja abiertos en pestañas: `Models/PurchaseOrder.cs` , `Data/MongoOrderRepository.cs` , `Application/OrderService.cs` , `Endpoints/OrdersEndpoint.cs` , `frontend/src/App.vue` .
3. En Netlify, agrega ya 1 producto al carrito.

1. Introducción (30 s)

QUÉ HACER: Muestra el editor con el proyecto abierto (carpeta `eshop-services` visible). No hace falta hacer clic todavía.

"Buenas, en este video voy a explicar la segunda fase del proyecto: la implementación de un tercer microservicio, Order.API, encargado de generar órdenes de compra a partir del carrito. A diferencia de Catalog.API y Basket.API, que persisten en PostgreSQL, este microservicio usa MongoDB Atlas como base de datos documental. Voy a explicar la arquitectura del código, y luego una demostración completa: desde agregar productos al carrito hasta consultar la orden generada, cambiarle el estado, descargarla en PDF, y verificar la persistencia directamente en MongoDB Atlas."

2. Arquitectura y código (4-5 min)

2.1 Vista general

QUÉ HACER: En el explorador de VS Code, expande `src/Order.API` para ver `Models` , `Data` , `Application` , `Endpoints` , `Services` .

"El microservicio sigue el mismo patrón que Catalog.API y Basket.API: ASP.NET Core Minimal API con Carter para el enrutamiento, pero con una separación clara de responsabilidades en cuatro capas."

2.2 Modelo de dominio

QUÉ HACER: Abre `Models/PurchaseOrder.cs` , señala `Id` , `CustomerId` , `CreatedAt` , `Status` , `Items` , `Subtotal` , `Tax` , `Total` . Cambia a `Models/OrderItem.cs` , señala `ProductId` , `Quantity` , `UnitPrice` .

"Primero, el dominio: la clase `PurchaseOrder` representa la orden, con un identificador único, el cliente, la fecha de creación, el estado, la lista de `OrderItem` , y los totales: subtotal, impuesto y total. Cada `orderItem` guarda el producto, la cantidad, y el precio unitario congelado al momento de la compra — no se recalcula después, aunque el precio del catálogo cambie."

2.3 Persistencia (MongoDB)

QUÉ HACER: Abre `Data/MongoOrderRepository.cs` . Señala el método `RunAsync` (línea ~22-34), luego el `catch (Exception ex) when (ex is MongoException or TimeoutException)` , y el constructor con `MongoClient(...)` .

"La capa de persistencia usa el driver oficial de MongoDB para .NET. Cada operación pasa por un método `RunAsync` que envuelve las llamadas al driver: si MongoDB no está disponible o hay un timeout, se captura la excepción interna del driver y se traduce a un error genérico. Esto es importante porque el driver por defecto expone detalles internos del clúster en el mensaje de error, y eso viola el requisito de no exponer información sensible al cliente. Aquí también hay un detalle técnico relevante: el driver de MongoDB en su versión 3 requiere declarar explícitamente cómo serializar los `Guid` , cosa que se configura una sola vez al inicio de `Program.cs` ."

2.4 Lógica de negocio

QUÉ HACER: Abre `Application/OrderService.cs` . Señala `AllowedTransitions` (línea ~25-29). En `CreateOrderAsync` , señala en orden: el bloque de `idempotencyKey` , la llamada a `basketClient.GetBasketAsync` , y el `foreach` con `catalogClient.ProductExistsAsync` .

"La capa de aplicación es donde vive la lógica de negocio. Al crear una orden, primero se revisa si ya existe una orden con el mismo `Idempotency-Key` para ese cliente — si existe, se devuelve esa misma orden en lugar de crear una nueva. Esto implementa idempotencia: si el cliente reintenta la misma solicitud, por ejemplo por un error de red, no se genera una compra duplicada. Después, se obtiene el carrito llamando por HTTP a `Basket.API`, se valida que no esté vacío, y por cada producto se valida que exista consultando a `Catalog.API`. Si algo falla en cualquiera de estas validaciones, se lanza una excepción que se traduce a `400 Bad Request` . Aquí también está la máquina de estados: un diccionario define las transiciones válidas — de `Pending` se puede pasar a `Confirmed` o a `Cancelled` , pero desde esos dos estados finales no se puede transicionar a ningún otro."

2.5 Endpoints

QUÉ HACER: Abre `Endpoints/OrdersEndpoint.cs` . Señala de arriba a abajo cada `group.MapPost/MapGet/MapPatch` .

"Finalmente, los endpoints: `POST /api/orders` para crear, `GET /api/orders` para listar todas, `GET /api/orders/{id}` para consultar una específica, `GET /api/orders/customer/{customerId}` para las de un cliente, y `PATCH /api/orders/{id}/status` para cambiar el estado. Todo documentado con Swagger."

2.6 Frontend

QUÉ HACER: Abre `frontend/src/App.vue` . Scroll rápido mostrando el `cart-drawer` , la vista `orders` y la vista `order` en el template. Vistazo general, sin detenerte mucho.

"Del lado del frontend, en Vue 3, agregué un carrito en un panel deslizable activado por un ícono, para no saturar la pantalla principal. Al confirmar la compra, se abre el detalle de la orden en una pestaña nueva, y desde ahí se puede cambiar el estado de la orden y descargar un comprobante en PDF generado en el navegador con la librería jsPDF. También agregué una vista de 'Pedidos' que lista todas las órdenes, con filtro por cliente y paginación."

3. Demo completa en producción (7-8 min)

3.1 Agregar al carrito y comprar

QUÉ HACER:

1. Cambia a la pestaña de Netlify (ya debería tener 1 producto en el carrito).
2. Clic en el ícono  arriba a la derecha para abrir el panel.
3. Clic en el botón azul "Realizar compra".
4. Se abre una pestaña nueva — cámbiate a ella.

"Ahora una demostración completa, todo contra el entorno de producción: frontend en Netlify, y los tres microservicios —Catalog, Basket y Order— desplegados en Render, cada uno con su propia base de datos en la nube. Al hacer clic en 'Realizar compra', el frontend llama a `POST /api/orders` con el `customerId`, el `basketId`, y un `Idempotency-Key` generado automáticamente. La orden se abre en una pestaña nueva, mostrando el identificador único, la fecha, la hora, los productos, y el estado inicial: `Pending`."

3.2 Verificar en MongoDB Atlas

QUÉ HACER:

1. Copia el ID de la orden en pantalla.
2. Cambia a la pestaña de MongoDB Atlas.
3. Menú izquierdo → "Clusters" → botón "Browse Collections" de tu cluster.
4. En la lista de bases, clic en `OrdersDb` → colección `orders`.
5. Busca el documento con ese ID (o el más reciente) y haz clic para expandirlo.

"Para comprobar que la persistencia es real y no solo en memoria, entro a MongoDB Atlas y busco el documento con el mismo identificador que acabamos de ver — aquí está, con exactamente los mismos datos: cliente, items, subtotal, impuesto, total y estado."

3.3 Cambiar el estado y descargar PDF

QUÉ HACER:

1. Regresa a la pestaña del detalle de la orden.
2. Clic en "Confirmar orden" — observa el badge cambiar de amarillo a verde.
3. Clic en  "Descargar PDF".
4. Abre el PDF descargado y muéstralo brevemente.

"Cambio el estado de la orden de `Pending` a `Confirmed`, usando el endpoint `PATCH /api/orders/{id}/status`, que valida que la transición sea permitida. Y descargo el comprobante en PDF, generado del lado del cliente con fecha, hora, productos, cantidades, precios y total."

3.4 Vista de Pedidos

QUÉ HACER:

1. Regresa a Netlify, catálogo principal.
2. Clic en  "Pedidos" (se abre pestaña nueva).
3. Muestra la tabla completa con scroll.
4. Escribe un nombre de cliente en "Filtrar por cliente" → clic "Buscar".
5. Clic en "Ver todas" para regresar al listado completo.

"En la vista de Pedidos se listan todas las órdenes generadas, con paginación, y puedo filtrar por cliente específico"

para ver solo sus compras."

3.5 Pruebas en Swagger

QUÉ HACER: Cambia a la pestaña de Swagger de Order.API.

"Para cerrar, algunas pruebas puntuales desde Swagger, la documentación interactiva de la API."

Prueba A — Carrito vacío (400 esperado):

QUÉ HACER:

1. Clic en la barra de `POST /api/orders` para expandirla.
2. Clic en "Try it out".
3. En el Request body escribe: `{ "customerId": "PruebaVacio", "basketId": "PruebaVacio" }`
4. Clic en "Execute".
5. Muestra el código 400 en "Server response".

"Primero, un carrito vacío: uso un cliente que nunca cargó nada al carrito. Responde `400 Bad Request`, como exige la regla de negocio."

Prueba B — Idempotencia:

QUÉ HACER:

1. En el mismo formulario, cambia el body al cliente real con carrito (agrega otro producto en Netlify si ya se vació).
2. En el campo `idempotencyKey` escribe: `prueba-video-1`
3. Clic en "Execute". Anota el `id` de la respuesta.
4. Sin cambiar nada, clic en "Execute" otra vez.
5. Compara: el `id` debe ser idéntico.

"Ahora, reenvío la misma solicitud con el mismo `Idempotency-Key`: en vez de crear una segunda orden, el sistema devuelve la orden que ya existía — el identificador es exactamente el mismo. Esto demuestra la idempotencia."

Prueba C — Transición de estado inválida:

QUÉ HACER:

1. Clic en la barra de `PATCH /api/orders/{id}/status` → "Try it out".
2. En `id`, pega el ID de una orden ya confirmada (la del paso 3.3).
3. Body: `{ "status": "Cancelled" }` → Execute (la cancela primero).
4. Con el mismo `id`, "Try it out" de nuevo, body: `{ "status": "Confirmed" }` → Execute.
5. Muestra el código 400.

"Y una transición de estado inválida, de `Cancelled` a `Confirmed`: el sistema la rechaza con `400 Bad Request`, protegiendo la integridad del ciclo de vida de la orden."

4. Cierre (30 s)

QUÉ HACER: Vuelve al editor o a Netlify en pantalla principal.

"En resumen: Order.API es un microservicio independiente con Minimal API, persistencia en MongoDB Atlas, idempotencia mediante `Idempotency-Key`, control de estados con transiciones validadas, e integración HTTP con los microservicios existentes de catálogo y carrito — todo publicado en la nube y funcionando en conjunto con el frontend. Gracias."

Checklist antes de grabar

- ☐ Las 3 pestañas del navegador abiertas y listas (Netlify, MongoDB Atlas, Swagger).
- ☐ Los 5 archivos del editor abiertos en pestañas separadas.
- ☐ Los 3 servicios en Render respondiendo.
- ☐ MongoDB Atlas con el cluster activo (no pausado — revisa antes, si dice "paused" dale Resume).
- ☐ Un producto ya agregado al carrito en Netlify antes de grabar.
- ☐ Contraseñas/connection strings tapadas si el video se comparte publicamente.